

# Terraform + GCP

## Learning Guide for Kubernetes Practitioners

---

*From kubectl apply -f to Infrastructure as Code*

July 2026

# Table of Contents

---

## Stage 0

What You're Actually Learning

## Stage 1

Setup (GCP + gcloud + Terraform)

## Stage 2

Your First Terraform Config

## Stage 3

Core Concepts (Variables, Outputs, State)

## Stage 4

Your First Real Resource - A GKE Cluster

## Stage 5

State Management (Critical for Real Use)

## Stage 6

Projects to Build (Not Tutorials)

## Your Next Steps

A Learning Schedule

## Stage 0: What You're Actually Learning

---

Right now you use `kubectl apply -f` to deploy YAML files manually. Terraform lets you describe your infrastructure - not just Kubernetes, but the whole stack: VPC, subnets, firewall rules, GKE clusters, databases - as code, version it in git, and apply it all with one command.

The mindset shift: You describe WHAT you want, not HOW to do it.

Tip: You already understand Kubernetes resources (Deployment, Service, Ingress). Terraform is the same concept, just for infrastructure instead of containers.

## Stage 1: Setup

---

### 1.1 Create a GCP Free Tier Account

Go to [cloud.google.com/free](https://cloud.google.com/free) and sign up. You get \$300 credit for 90 days, plus always-free tier resources. A credit card is required for identity verification, but you won't be charged as long as you stay within the free tier.

### 1.2 Install gcloud CLI

Open Terminal and run:

```
brew install --cask google-cloud-sdk
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
```

### 1.3 Install Terraform

```
brew install terraform
terraform --version    # should show v1.x
```

### 1.4 Create a GCP Project

```
gcloud projects create YOUR-PROJECT-NAME \
  --name="Learning Terraform"
gcloud config set project YOUR-PROJECT-NAME
```

## Stage 2: Your First Terraform Config

---

Don't read the docs yet. Just follow along and see it work.

### 2.1 Create a Project Folder

```
mkdir ~/learn-terraform && cd ~/learn-terraform
```

### 2.2 Create main.tf

Create a file called main.tf with this content:

```
terraform {
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 6.0"
    }
  }
}

provider "google" {
  project = "YOUR-PROJECT-ID"
  region  = "asia-southeast2"
}

resource "google_storage_bucket" "my_first_bucket" {
  name          = "YOUR-PROJECT-ID-baby-bucket"
  location      = "ASIA-SOUTHEAST2"
  force_destroy = true
}
```

### 2.3 Run the Terraform Workflow

Initialize - downloads provider plugins:

```
terraform init
```

Preview - see what will be created (always do this first):

```
terraform plan
```

Apply - create it for real (type 'yes' when prompted):

```
terraform apply
```

Check in GCP Console under Cloud Storage. You should see your bucket.

### 2.4 Destroy It

This is important - you can experiment freely knowing nothing is permanent:

```
terraform destroy
```

Why teach destroy first? Because you'll experiment freely. Bucket costs pennies, but the habit of destroying unused resources saves real money on bigger projects.

## Stage 3: Core Concepts

---

### Variables (don't hardcode your project ID)

Create variables.tf:

```
variable "project_id" {
  description = "Your GCP project ID"
  type        = string
}

variable "region" {
  description = "GCP region"
  type        = string
  default     = "asia-southeast2"
}
```

Update main.tf to use variables:

```
provider "google" {
  project = var.project_id
  region  = var.region
}

resource "google_storage_bucket" "bucket" {
  name          = "${var.project_id}-data"
  location      = var.region
  force_destroy = true
}
```

Create terraform.tfvars (your actual values go here):

```
project_id = "your-actual-project-id"
```

**Warning:** Add terraform.tfvars to .gitignore if it contains secrets. terraform.tfvars is NOT committed to git in production.

Run:

```
terraform plan
```

```
terraform apply
```

### Outputs

Create outputs.tf:

```
output "bucket_url" {
  value = google_storage_bucket.bucket.url
}
```

After 'terraform apply', the bucket URL prints to screen. Also run:

```
terraform output
```

### State

Look at terraform.tfstate - it's JSON. This file is critical. It maps your config to real cloud resources. If you lose it,

Terraform doesn't know what it manages.

Never edit terraform.tfstate by hand. Never delete it. If working with a team, store it in GCS (see Stage 5).

## Stage 4: Your First Real Resource - A GKE Cluster

This is where your Kubernetes knowledge connects. You'll create a GKE cluster with Terraform, then deploy your existing apps to it.

### 4.1 Enable the GKE API

```
gcloud services enable container.googleapis.com
```

### 4.2 Create gke.tf

```
resource "google_container_cluster" "learning" {
  name     = "learning-cluster"
  location = var.region

  initial_node_count = 1
  deletion_protection = false
}

resource "google_container_node_pool" "primary_nodes" {
  name       = "primary"
  location   = var.region
  cluster    = google_container_cluster.learning.name
  node_count = 1

  node_config {
    machine_type = "e2-small"
    disk_size_gb = 20
    preemptible  = true
  }
}
```

Explanation:

preemptible = true means ~60% cheaper cost, but GCP can terminate the instance anytime. Perfect for learning - you pay almost nothing.

### 4.3 Apply It

```
terraform apply
```

This takes 3-5 minutes. Go make tea.

### 4.4 Connect to Your Cluster

```
gcloud container clusters get-credentials learning-cluster \
  --region asia-southeast2
kubectl get nodes
```

You'll see 1 node. This is YOUR Kubernetes cluster.

Run 'kubectl apply -f' against your existing k8s/ YAML files to deploy your apps. The same manifests work on your new GKE cluster.

## Stage 5: State Management

---

Right now your terraform.tfstate is on your laptop. If your hard drive dies, Terraform can't manage your infrastructure anymore.

### 5.1 Create a GCS Bucket for State

First, make sure the bucket from Stage 2 still exists, then apply again:

```
terraform apply
```

### 5.2 Add Backend Configuration

Create backend.tf:

```
terraform {  
  backend "gcs" {  
    bucket = "YOUR-PROJECT-ID-tfstate"  
    prefix = "gke-cluster"  
  }  
}
```

### 5.3 Migrate State to GCS

```
terraform init -migrate-state
```

Type 'yes' to copy the local state file to GCS.

Now your state is stored in the cloud. Delete your local terraform.tfstate - you don't need it anymore. When you or a teammate runs 'terraform init', it pulls the latest state from the bucket.

## Stage 6: Projects to Build

---

### Project 1: Deploy cert-manager with Terraform

Instead of applying clusterissuer.yaml manually, use Terraform's Helm provider to install cert-manager and create the ClusterIssuer.

```
resource "helm_release" "cert_manager" {
  name      = "cert-manager"
  repository = "https://charts.jetstack.io"
  chart     = "cert-manager"
  namespace = "cert-manager"

  set {
    name = "installCRDs"
    value = "true"
  }
}
```

### Project 2: Custom VPC + Subnetwork + GKE Cluster

Your cluster currently uses the default VPC. Create a custom one with private subnets, NAT gateway, and firewall rules.

### Project 3: Deploy Your Apps Entirely Through Terraform

Use the Kubernetes provider (hashicorp/kubernetes) to define Deployments, Services, and Ingresses as Terraform resources. Now your entire stack is IaC.

```
resource "kubernetes_deployment_v1" "megumi" {
  metadata {
    name = "megumi"
  }
  spec {
    replicas = 1
    selector {
      match_labels = {
        app = "megumi"
      }
    }
  }
  template {
    metadata {
      labels = { app = "megumi" }
    }
    spec {
      container {
        image = "ghcr.io/your/megumi:v1.0.0"
        name  = "megumi"
      }
    }
  }
}
```

## Your Next Steps

---

Don't try to learn everything at once. Follow this schedule:

### This Week

- Stage 1: Setup GCP, gcloud, Terraform
- Stage 2: Create and destroy your first bucket
- Stage 3: Add variables, outputs, understand state

### Next Week

- Stage 4: Create a GKE cluster, deploy your apps to it
- Destroy and recreate the cluster 2-3 times until the workflow feels automatic

### Two Weeks

- Stage 5: Set up remote state in GCS
- Stage 6, Project 1: Deploy cert-manager with Terraform

### Month 2

- Stage 6, Project 2: Custom VPC with private GKE cluster
- Stage 6, Project 3: Convert one app's full stack to Terraform

## Golden Rules

---

### Always 'terraform plan' before 'terraform apply'

Never skip it. The plan shows you EXACTLY what Terraform will create, change, or destroy.

### One directory per project

Don't put everything in one monolith. Create a directory per project or stack.

### Destroy what you don't use

GCP bills by the minute. Always 'terraform destroy' after a learning session if you won't use it for a while.

### Never edit .tfstate manually

If something is wrong, fix it in the .tf files and run terraform apply. Directly editing state will corrupt it.